

TRABALHO DE CONCLUSÃO DA DISCIPLINA – TCD

UniCeub Campus Asa Norte

Faculdade de Tecnologia e Ciências Sociais - FATECS

Curso: Ciência da Computação

Disciplina: Compiladores

Professor: Leonardo Pol

TRABALHO DE CONCLUSÃO DA DISCIPLINA - TCD

AValiação FINAL - Laboratório de Pesquisa de Compiladores

Projeto B - Compilador para a Linguagem LPS1

Alunos: Davi Maia, Cayo Zickler e Nathan

Turma: B

Itens entregues:

Parte (a) - Analisador sintático + geração de código C misturada ao parser

Parte (b) - Analisador sintático + ASA + geração de código C nos métodos da ASA

Exemplo 1 em LPS1 e saída C gerada

Exemplo 2 em LPS1 e saída C gerada

2. Arquivos da Parte Comum

TokenType.java

```
package lps1.common;

public enum TokenType {
    SYMBOL,
    VARIABLE,
    NUMBER,
    EOF
}
```

Token.java

```
package lps1.common;

public final class Token {
    private final TokenType type;
    private final String text;
    private final int line;
    private final int column;

    public Token(TokenType type, String text, int line, int column) {
        this.type = type;
        this.text = text;
        this.line = line;
        this.column = column;
    }

    public TokenType getType() {
        return type;
    }

    public String getText() {
        return text;
    }

    public int getLine() {
        return line;
    }

    public int getColumn() {
        return column;
    }

    @Override
    public String toString() {
        if (type == TokenType.EOF) {
            return "fim do arquivo";
        }
        return "'" + text + "'";
    }
}
```

CompileException.java

```
package lps1.common;

public class CompileException extends RuntimeException {
    public CompileException(String message) {
        super(message);
    }
}
```

Lexer.java

```
package lps1.common;

/**
 * Analisador lexico da linguagem LPS1.
 *
 * Regras consideradas:
 * - Variavel: uma letra minuscula de 'a' ate 'z'.
 * - Numero: um digito de '0' ate '9'.
 * - Simbolos/comandos: = G + - * / % P I W { } < #.
 * - Espacos, quebras de linha e tabulacoes sao ignorados.
 */
public final class Lexer {
    private final String input;
    private int index;
    private int line;
    private int column;

    public Lexer(String input) {
        this.input = input == null ? "" : input;
        this.index = 0;
        this.line = 1;
        this.column = 1;
    }

    public Token nextToken() {
        skipWhitespace();

        if (index >= input.length()) {
            return new Token(TokenType.EOF, "", line, column);
        }

        char ch = input.charAt(index);
        int tokenLine = line;
        int tokenColumn = column;
        advance();

        if (ch >= 'a' && ch <= 'z') {
            return new Token(TokenType.VARIABLE, String.valueOf(ch), tokenLine, tokenColumn);
        }

        if (ch >= '0' && ch <= '9') {
            return new Token(TokenType.NUMBER, String.valueOf(ch), tokenLine, tokenColumn);
        }

        if (isSymbol(ch)) {
            return new Token(TokenType.SYMBOL, String.valueOf(ch), tokenLine, tokenColumn);
        }

        throw new CompileException(
            "Erro lexico na linha " + tokenLine + ", coluna " + tokenColumn +
            ": esperado variavel minuscula, numero ou simbolo valido, encontrado '" + ch + "'");
    }

    private boolean isSymbol(char ch) {
        return ch == '=' || ch == 'G' || ch == '+' || ch == '-' || ch == '*' ||
            ch == '/' || ch == '%' || ch == 'P' || ch == 'I' || ch == 'W' ||
            ch == '{' || ch == '}' || ch == '<' || ch == '#';
    }

    private void skipWhitespace() {
        while (index < input.length()) {
            char ch = input.charAt(index);
            if (Character.isWhitespace(ch)) {
                advance();
            } else {
                break;
            }
        }
    }

    private void advance() {
        if (index >= input.length()) {
            return;
        }

        char ch = input.charAt(index);
        index++;

        if (ch == '\n') {
            line++;
        }
    }
}
```

```
column = 1;  
} else {  
column++;  
}  
}  
}
```

CWriter.java

```
package lps1.common;

/**
 * Pequeno utilitario para gerar codigo C indentado.
 */
public final class CWriter {
    private final StringBuilder out = new StringBuilder();
    private int indent = 0;

    public void emitLine(String line) {
        for (int i = 0; i < indent; i++) {
            out.append(" ");
        }
        out.append(line).append(System.lineSeparator());
    }

    public void indent() {
        indent++;
    }

    public void dedent() {
        if (indent > 0) {
            indent--;
        }
    }

    public String getCode() {
        return out.toString();
    }

    public static String wrapMain(String body) {
        StringBuilder code = new StringBuilder();
        code.append("#include <stdio.h>\n");
        code.append("\n");
        code.append("int main() {\n");
        code.append("    int a, b, c, d, e, f, g, h, i, j, k, l, m, n, o, p, q, r, s, t, u, v, w, x, y, z;\n");
        code.append("    char str[512]; // auxiliar na leitura com G\n");
        if (body != null && !body.isEmpty()) {
            code.append(body);
        }
        code.append("}\n");
        return code.toString();
    }
}
```

FileUtil.java

```
package lps1.common;

import java.io.IOException;
import java.io.ByteArrayOutputStream;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Paths;

public final class FileUtil {
    private FileUtil() {}

    public static String readInput(String[] args) throws IOException {
        if (args.length > 0) {
            return new String(Files.readAllBytes(Paths.get(args[0])), StandardCharsets.UTF_8);
        }

        ByteArrayOutputStream buffer = new ByteArrayOutputStream();
        byte[] data = new byte[4096];
        int bytesRead;
        while ((bytesRead = System.in.read(data)) != -1) {
            buffer.write(data, 0, bytesRead);
        }
        return buffer.toString(StandardCharsets.UTF_8.name());
    }
}
```

3. Arquivos da Parte (a)

CompiladorA.java

```
package lps1.a;

import lps1.common.CompileException;
import lps1.common.FileUtil;

/**
 * PARTE (a)
 * Programa principal: le código LPS1 de um arquivo ou da entrada padrão
 * e imprime o código C gerado na saída padrão.
 */
public final class CompiladorA {
    private CompiladorA() {}

    public static void main(String[] args) {
        try {
            String source = FileUtil.readInput(args);
            String cCode = new ParserA(source).compile();
            System.out.println(cCode);
        } catch (CompileException ex) {
            System.err.println(ex.getMessage());
            System.exit(1);
        } catch (Exception ex) {
            System.err.println("Erro ao ler/compilar o arquivo: " + ex.getMessage());
            System.exit(1);
        }
    }
}
```

ParserA.java

```
package lps1.a;

import lps1.common.CWriter;
import lps1.common.CompileException;
import lps1.common.Lexer;
import lps1.common.Token;
import lps1.common.TokenType;

/**
 * PARTE (a)
 * Analisador sintatico descendente recursivo com geracao de codigo C
 * misturada aos metodos do parser.
 */
public final class ParserA {
    private final Lexer lexer;
    private final CWriter writer;
    private Token current;

    public ParserA(String source) {
        this.lexer = new Lexer(source);
        this.writer = new CWriter();
        this.current = lexer.nextToken();
        this.writer.indent(); // corpo dentro de int main()
    }

    public String compile() {
        parseProgram();
        expectEOF();
        return CWriter.wrapMain(writer.getCode());
    }

    private void parseProgram() {
        if (isEOF()) {
            error("comando");
        }

        while (!isEOF()) {
            if (isSymbol("{}")) {
                error("comando antes de '{}')");
            }
            parseCommand();
        }
    }

    private void parseCommand() {
        if (isSymbol("=")) {
            parseAssignCommand();
        } else if (isSymbol("G")) {
            parseGetCommand();
        } else if (isSymbol("+")) {
            parseBinaryCommand("+", "+");
        } else if (isSymbol("-")) {
            parseBinaryCommand("-", "-");
        } else if (isSymbol("*")) {
            parseBinaryCommand("*", "*");
        } else if (isSymbol("/")) {
            parseBinaryCommand("/", "/");
        } else if (isSymbol("%")) {
            parseBinaryCommand("%", "%");
        } else if (isSymbol("P")) {
            parsePrintCommand();
        } else if (isSymbol("I")) {
            parseIfCommand();
        } else if (isSymbol("W")) {
            parseWhileCommand();
        } else if (isSymbol("{")) {
            parseCompositeCommandAsStandaloneBlock();
        } else {
            error("comando valido (=, G, +, -, *, /, %, P, I, W ou {)");
        }
    }

    private void parseAssignCommand() {
        expectSymbol("=");
        String variable = expectVariable();
        String value = parseValue();
        writer.emitLine(variable + " = " + value + ";");
    }

    private void parseGetCommand() {
```

```

expectSymbol("6");
String variable = expectVariable();
writer.emitLine("{");
writer.indent();
writer.emitLine("gets(str);");
writer.emitLine("scanf(str, \"%d\", &" + variable + ");");
writer.dedent();
writer.emitLine("}");
}

private void parseBinaryCommand(String commandSymbol, String cOperator) {
expectSymbol(commandSymbol);
String variable = expectVariable();
String left = parseValue();
String right = parseValue();
writer.emitLine(variable + " = " + left + " " + cOperator + " " + right + ";");
}

private void parsePrintCommand() {
expectSymbol("P");
String value = parseValue();
writer.emitLine("printf(\"%d\\n\", " + value + ");");
}

private void parseIfCommand() {
expectSymbol("I");
String comparison = parseComparison();
writer.emitLine("if ( " + comparison + " ) {");
writer.indent();
parseControlBody();
writer.dedent();
writer.emitLine("}");
}

private void parseWhileCommand() {
expectSymbol("W");
String comparison = parseComparison();
writer.emitLine("while ( " + comparison + " ) {");
writer.indent();
parseControlBody();
writer.dedent();
writer.emitLine("}");
}

/**
 * Corpo de I/W. Se a LPS1 trouxe um comando composto { ... }, usa os
 * comandos internos dentro do bloco C ja aberto pelo if/while. Se trouxe
 * comando simples, gera apenas esse comando dentro das chaves do if/while.
 */
private void parseControlBody() {
if (isSymbol("{")) {
expectSymbol("{");
if (isSymbol("}")) {
error("comando dentro do bloco composto");
}
while (!isSymbol("}")) {
if (isEOF()) {
error("'}'");
}
parseCommand();
}
expectSymbol("}");
} else {
parseCommand();
}
}

/**
 * Comando composto quando aparece como comando independente, fora de I/W.
 */
private void parseCompositeCommandAsStandaloneBlock() {
expectSymbol("{");
writer.emitLine("{");
writer.indent();
if (isSymbol("}")) {
error("comando dentro do bloco composto");
}
while (!isSymbol("}")) {
if (isEOF()) {
error("'}'");
}
parseCommand();
}
}

```

```

}
expectSymbol("}");
writer.dedent();
writer.emitLine("}");
}

private String parseComparison() {
String variable = expectVariable();
String operator = parseOperator();
String value = parseValue();
return variable + " " + operator + " " + value;
}

private String parseOperator() {
if (isSymbol("=")) {
advance();
return "=";
}
if (isSymbol("<")) {
advance();
return "<";
}
if (isSymbol("#")) {
advance();
return "!=";
}
error("operador de comparacao (=, < ou #)");
return ""; // inalcançavel
}

private String parseValue() {
if (current.getType() == TokenType.VARIABLE || current.getType() == TokenType.NUMBER) {
String value = current.getText();
advance();
return value;
}
error("valor (variavel ou numero)");
return ""; // inalcançavel
}

private String expectVariable() {
if (current.getType() != TokenType.VARIABLE) {
error("variavel (letra minuscula)");
}
String variable = current.getText();
advance();
return variable;
}

private void expectSymbol(String symbol) {
if (!isSymbol(symbol)) {
error("'" + symbol + "'");
}
advance();
}

private void expectEOF() {
if (!isEOF()) {
error("fim do arquivo");
}
}

private boolean isSymbol(String symbol) {
return current.getType() == TokenType.SYMBOL && current.getText().equals(symbol);
}

private boolean isEOF() {
return current.getType() == TokenType.EOF;
}

private void advance() {
current = lexer.nextToken();
}

private void error(String expected) {
throw new CompileException(
"Erro sintatico na linha " + current.getLine() + ", coluna " + current.getColumn() +
": esperado " + expected + ", encontrado " + current + ".");
}
}

```

4. Arquivos da Parte (b)

CompiladorB.java

```
package lps1.b;

import lps1.b.ASA.ProgramNode;
import lps1.common.CompileException;
import lps1.common.FileUtil;

/**
 * PARTE (b)
 * Programa principal: le código LPS1, constroi a ASA e imprime o código C
 * gerado pelos métodos da própria ASA.
 */
public final class CompiladorB {
    private CompiladorB() {}

    public static void main(String[] args) {
        try {
            String source = FileUtil.readInput(args);
            ProgramNode program = new ParserB(source).parseProgram();
            System.out.println(program.toCCode());
        } catch (CompileException ex) {
            System.err.println(ex.getMessage());
            System.exit(1);
        } catch (Exception ex) {
            System.err.println("Erro ao ler/compilar o arquivo: " + ex.getMessage());
            System.exit(1);
        }
    }
}
```

ParserB.java

```
package lps1.b;

import java.util.ArrayList;
import java.util.List;

import lps1.b.ASA.AssignCommandNode;
import lps1.b.ASA.BinaryCommandNode;
import lps1.b.ASA.CommandNode;
import lps1.b.ASA.ComparisonNode;
import lps1.b.ASA.CompositeCommandNode;
import lps1.b.ASA.GetCommandNode;
import lps1.b.ASA.IfCommandNode;
import lps1.b.ASA.PrintCommandNode;
import lps1.b.ASA.ProgramNode;
import lps1.b.ASA.ValueNode;
import lps1.b.ASA.WhileCommandNode;
import lps1.common.CompileException;
import lps1.common.Lexer;
import lps1.common.Token;
import lps1.common.TokenType;

/**
 * PARTE (b)
 * Analisador sintatico descendente recursivo que constroi a ASA.
 * A geracao de C nao aparece aqui: fica nas classes de ASA.java.
 */
public final class ParserB {
    private final Lexer lexer;
    private Token current;

    public ParserB(String source) {
        this.lexer = new Lexer(source);
        this.current = lexer.nextToken();
    }

    public ProgramNode parseProgram() {
        if (isEOF()) {
            error("comando");
        }

        List<CommandNode> commands = new ArrayList<>();
        while (!isEOF()) {
            if (isSymbol("}")) {
                error("comando antes de '}'");
            }
            commands.add(parseCommand());
        }
        expectEOF();
        return new ProgramNode(commands);
    }

    private CommandNode parseCommand() {
        if (isSymbol("=")) {
            return parseAssignCommand();
        } else if (isSymbol("G")) {
            return parseGetCommand();
        } else if (isSymbol("+")) {
            return parseBinaryCommand("+", "+");
        } else if (isSymbol("-")) {
            return parseBinaryCommand("-", "-");
        } else if (isSymbol("*")) {
            return parseBinaryCommand("*", "*");
        } else if (isSymbol("/")) {
            return parseBinaryCommand("/", "/");
        } else if (isSymbol("%")) {
            return parseBinaryCommand("%", "%");
        } else if (isSymbol("P")) {
            return parsePrintCommand();
        } else if (isSymbol("I")) {
            return parseIfCommand();
        } else if (isSymbol("W")) {
            return parseWhileCommand();
        } else if (isSymbol("{")) {
            return parseCompositeCommand();
        } else {
            error("comando valido (=, G, +, -, *, /, %, P, I, W ou {)");
            return null; // inalcançavel
        }
    }
}
```

```

private CommandNode parseAssignCommand() {
    expectSymbol("=");
    String variable = expectVariable();
    ValueNode value = parseValue();
    return new AssignCommandNode(variable, value);
}

private CommandNode parseGetCommand() {
    expectSymbol("G");
    String variable = expectVariable();
    return new GetCommandNode(variable);
}

private CommandNode parseBinaryCommand(String commandSymbol, String cOperator) {
    expectSymbol(commandSymbol);
    String variable = expectVariable();
    ValueNode left = parseValue();
    ValueNode right = parseValue();
    return new BinaryCommandNode(variable, left, cOperator, right);
}

private CommandNode parsePrintCommand() {
    expectSymbol("P");
    ValueNode value = parseValue();
    return new PrintCommandNode(value);
}

private CommandNode parseIfCommand() {
    expectSymbol("I");
    ComparisonNode comparison = parseComparison();
    CommandNode body = parseCommand();
    return new IfCommandNode(comparison, body);
}

private CommandNode parseWhileCommand() {
    expectSymbol("W");
    ComparisonNode comparison = parseComparison();
    CommandNode body = parseCommand();
    return new WhileCommandNode(comparison, body);
}

private CompositeCommandNode parseCompositeCommand() {
    expectSymbol("{");
    if (isSymbol("}")) {
        error("comando dentro do bloco composto");
    }

    List<CommandNode> commands = new ArrayList<>();
    while (!isSymbol("}")) {
        if (isEOF()) {
            error("' '");
        }
        commands.add(parseCommand());
    }
    expectSymbol("}");
    return new CompositeCommandNode(commands);
}

private ComparisonNode parseComparison() {
    String variable = expectVariable();
    String operator = parseOperator();
    ValueNode value = parseValue();
    return new ComparisonNode(variable, operator, value);
}

private String parseOperator() {
    if (isSymbol("=")) {
        advance();
        return "==";
    }
    if (isSymbol("<")) {
        advance();
        return "<";
    }
    if (isSymbol("#")) {
        advance();
        return "!=";
    }
    error("operador de comparacao (=, < ou #)");
    return ""; // inalcançavel
}

```

```

private ValueNode parseValue() {
    if (current.getType() == TokenType.VARIABLE || current.getType() == TokenType.NUMBER) {
        String text = current.getText();
        advance();
        return new ValueNode(text);
    }
    error("valor (variavel ou numero)");
    return null; // inalcançavel
}

private String expectVariable() {
    if (current.getType() != TokenType.VARIABLE) {
        error("variavel (letra minuscula)");
    }
    String variable = current.getText();
    advance();
    return variable;
}

private void expectSymbol(String symbol) {
    if (!isSymbol(symbol)) {
        error("'" + symbol + "'");
    }
    advance();
}

private void expectEOF() {
    if (!isEOF()) {
        error("fim do arquivo");
    }
}

private boolean isSymbol(String symbol) {
    return current.getType() == TokenType.SYMBOL && current.getText().equals(symbol);
}

private boolean isEOF() {
    return current.getType() == TokenType.EOF;
}

private void advance() {
    current = lexer.nextToken();
}

private void error(String expected) {
    throw new CompileException(
        "Erro sintatico na linha " + current.getLine() + ", coluna " + current.getColumn() +
        ": esperado " + expected + ", encontrado " + current + "."
    );
}

```

ASA.java

```
package lps1.b;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

import lps1.common.CWriter;

/**
 * PARTE (b)
 * ASA = Arvore Sintatica Abstrata da LPS1.
 * A geracao de codigo C fica nos metodos das classes da ASA.
 */
public final class ASA {
    private ASA() {}

    public abstract static class Node {
        public abstract void generateC(CWriter writer);
    }

    public static final class ProgramNode extends Node {
        private final List<CommandNode> commands;

        public ProgramNode(List<CommandNode> commands) {
            this.commands = Collections.unmodifiableList(new ArrayList<>(commands));
        }

        @Override
        public void generateC(CWriter writer) {
            writer.indent();
            for (CommandNode command : commands) {
                command.generateC(writer);
            }
            writer.dedent();
        }

        public String toCCode() {
            CWriter writer = new CWriter();
            generateC(writer);
            return CWriter.wrapMain(writer.getCode());
        }
    }

    public abstract static class CommandNode extends Node {}

    public static final class ValueNode extends Node {
        private final String text;

        public ValueNode(String text) {
            this.text = text;
        }

        public String asC() {
            return text;
        }
    }

    @Override
    public void generateC(CWriter writer) {
        writer.emitLine(text);
    }
}

    public static final class ComparisonNode extends Node {
        private final String variable;
        private final String operator;
        private final ValueNode value;

        public ComparisonNode(String variable, String operator, ValueNode value) {
            this.variable = variable;
            this.operator = operator;
            this.value = value;
        }

        public String asC() {
            return variable + " " + operator + " " + value.asC();
        }
    }

    @Override
```

```

public void generateC(CWriter writer) {
writer.emitLine(asC());
}
}

public static final class AssignCommandNode extends CommandNode {
private final String variable;
private final ValueNode value;

public AssignCommandNode(String variable, ValueNode value) {
this.variable = variable;
this.value = value;
}

@Override
public void generateC(CWriter writer) {
writer.emitLine(variable + " = " + value.asC() + ";");
}
}

public static final class GetCommandNode extends CommandNode {
private final String variable;

public GetCommandNode(String variable) {
this.variable = variable;
}

@Override
public void generateC(CWriter writer) {
writer.emitLine("{");
writer.indent();
writer.emitLine("gets(str);");
writer.emitLine("scanf(str, \"%d\\", &" + variable + ");");
writer.dedent();
writer.emitLine("}");
}
}

public static final class BinaryCommandNode extends CommandNode {
private final String variable;
private final ValueNode left;
private final String operator;
private final ValueNode right;

public BinaryCommandNode(String variable, ValueNode left, String operator, ValueNode right) {
this.variable = variable;
this.left = left;
this.operator = operator;
this.right = right;
}

@Override
public void generateC(CWriter writer) {
writer.emitLine(variable + " = " + left.asC() + " " + operator + " " + right.asC() + ";");
}
}

public static final class PrintCommandNode extends CommandNode {
private final ValueNode value;

public PrintCommandNode(ValueNode value) {
this.value = value;
}

@Override
public void generateC(CWriter writer) {
writer.emitLine("printf(\"%d\\n\\", " + value.asC() + ");");
}
}

public static final class IfCommandNode extends CommandNode {
private final ComparisonNode comparison;
private final CommandNode body;

public IfCommandNode(ComparisonNode comparison, CommandNode body) {
this.comparison = comparison;
this.body = body;
}

@Override
public void generateC(CWriter writer) {
writer.emitLine("if ( " + comparison.asC() + " ) {");
}
}

```

```

writer.indent();
emitControlBody(writer, body);
writer.dedent();
writer.emitLine("}");
}
}

public static final class WhileCommandNode extends CommandNode {
private final ComparisonNode comparison;
private final CommandNode body;

public WhileCommandNode(ComparisonNode comparison, CommandNode body) {
this.comparison = comparison;
this.body = body;
}

@Override
public void generateC(CWriter writer) {
writer.emitLine("while ( " + comparison.asC() + " ) {");
writer.indent();
emitControlBody(writer, body);
writer.dedent();
writer.emitLine("}");
}
}

public static final class CompositeCommandNode extends CommandNode {
private final List<CommandNode> commands;

public CompositeCommandNode(List<CommandNode> commands) {
this.commands = Collections.unmodifiableList(new ArrayList<>(commands));
}

@Override
public void generateC(CWriter writer) {
writer.emitLine("{");
writer.indent();
generateContents(writer);
writer.dedent();
writer.emitLine("}");
}

private void generateContents(CWriter writer) {
for (CommandNode command : commands) {
command.generateC(writer);
}
}
}

private static void emitControlBody(CWriter writer, CommandNode body) {
if (body instanceof CompositeCommandNode) {
((CompositeCommandNode) body).generateContents(writer);
} else {
body.generateC(writer);
}
}
}

```

5. Exemplos LPS1

exemplo1.lps1

```
G n
G p
= i 0
W i # n {
* a p i
P a
+ i i 1
}
```

exemplo2.lps1

```
G n
= i 2
% a n i
W i < n {
I a = 0 = i n
+ i i 1
% a n i
}
I a = 0 P 0
I a # 0 P 1
```

6. Código C Gerado

exemplo1_parte_a.c

```
#include <stdio.h>

int main() {
    int a, b, c, d, e, f, g, h, i, j, k, l, m, n, o, p, q, r, s, t, u, v, w, x, y, z;
    char str[512]; // auxiliar na leitura com G
    {
        gets(str);
        sscanf(str, "%d", &n);
    }
    {
        gets(str);
        sscanf(str, "%d", &p);
    }
    i = 0;
    while ( i != n ) {
        a = p * i;
        printf("%d\n", a);
        i = i + 1;
    }
}
```

exemplo2_parte_a.c

```
#include <stdio.h>

int main() {
    int a, b, c, d, e, f, g, h, i, j, k, l, m, n, o, p, q, r, s, t, u, v, w, x, y, z;
    char str[512]; // auxiliar na leitura com G
    {
        gets(str);
        sscanf(str, "%d", &n);
    }
    i = 2;
    a = n % i;
    while ( i < n ) {
        if ( a == 0 ) {
            i = n;
        }
        i = i + 1;
        a = n % i;
    }
    if ( a == 0 ) {
        printf("%d\n", 0);
    }
    if ( a != 0 ) {
        printf("%d\n", 1);
    }
}
```

exemplo1_parte_b.c

```
#include <stdio.h>

int main() {
    int a, b, c, d, e, f, g, h, i, j, k, l, m, n, o, p, q, r, s, t, u, v, w, x, y, z;
    char str[512]; // auxiliar na leitura com G
    {
        gets(str);
        sscanf(str, "%d", &n);
    }
    {
        gets(str);
        sscanf(str, "%d", &p);
    }
    i = 0;
    while ( i != n ) {
        a = p * i;
        printf("%d\n", a);
        i = i + 1;
    }
}
```

exemplo2_parte_b.c

```
#include <stdio.h>

int main() {
int a, b, c, d, e, f, g, h, i, j, k, l, m, n, o, p, q, r, s, t, u, v, w, x, y, z;
char str[512]; // auxiliar na leitura com G
{
gets(str);
sscanf(str, "%d", &n);
}
i = 2;
a = n % i;
while ( i < n ) {
if ( a == 0 ) {
i = n;
}
i = i + 1;
a = n % i;
}
if ( a == 0 ) {
printf("%d\n", 0);
}
if ( a != 0 ) {
printf("%d\n", 1);
}
}
```

Resposta da Pergunta: O que faz o segundo exemplo?

O segundo exemplo testa se o número lido é primo ou composto. Ele procura um divisor entre 2 e $n - 1$. Na lógica do exemplo, imprime **0** para número primo e **1** para número composto.